

61. Le panneau, et les nuances

Un score qui reste en place pendant que le décor glisse.

1. Ce qu'il faut comprendre

Le décor défile ; le score, lui, ne doit pas bouger. Sans une seconde couche, il faudrait le redessiner à chaque image à la position que le défilement lui a donnée — et le moindre décalage se verrait.

Le **panneau** est cette couche : le matériel l'appelle la fenêtre, il la pose par-dessus le décor, et **elle ne défile pas**. `panneau(x, y)` la place, en pixels, et l'allume. On y écrit avec `textePanneau()` et `poserPanneau()`, exactement comme sur le fond.

Il n'y a que **deux cartes de fond** dans la console : le décor prend la première, le panneau la seconde. C'est un choix, et il vaut mieux le savoir que le découvrir.

`paletteFond(n0, n1, n2, n3)` choisit les quatre nuances, de la plus claire à la plus sombre — chaque nombre va de 0 à 3. Elles étaient posées une fois pour toutes à l'allumage ; ici on s'en sert pour assombrir le fond par paliers.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <Tuile>` → [0.107.2. Tuile — dessiner une case](#)
- `#include <poser>` → [0.1.3. La fonction poser\(\) — poser une tuile](#)
- `#include <textePanneau>` → [60.2. La fonction textePanneau\(\) — écrire dans le panneau](#)
- `#include <panneau>` → [60.1. La fonction panneau\(\) — montrer le panneau](#)
- `#include <defiler>` → [0.90.3. La fonction defiler\(\) — faire glisser le fond](#)
- `#include <paletteFond>` → [60.6. La fonction paletteFond\(\) — les quatre nuances du fond](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **533 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage      58 o ← la console, à l'allumage - la mise en route de la console
main()         181 o ← démarrage - ta fonction
VBlank         8 o ← la console, à chaque image - compte les images, 60 fois par seconde
AttendreImage  28 o ← image() - attend l'image suivante
AttendreVBlank 16 o ← démarrage - attend que l'écran soit libre
AttendreAcces  12 o ← EcrireTexte - attend de pouvoir écrire une case
AdresseCase    10 o ← poser() - trouve l'adresse d'une case de l'écran
EcrireTexte    10 o ← textePanneau() - écrit un texte, lettre par lettre
CopierTuiles   76 o ← démarrage - copie les lettres et les dessins en mémoire vidéo
EffacerCarte   14 o ← démarrage - vide les deux cartes de l'écran
Tuiles         112 o ← CopierTuiles - 6 lettres sur 43, et 1 tuile dessinée
"SCORE 00"     8 o ← textePanneau() - un texte gravé
```

Lettres gravées : CEORS0

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, EffacerCases, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, LireManette,

DonneesTransfert, Notes

Fonctions du langage appelées :

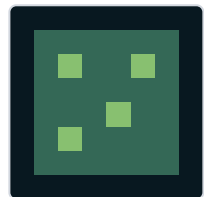
paletteFond()	x 1	61 o sur place, dans main()	
poser()	x 2	26 o sur place, dans main()	+ AdresseCase (10 o)
panneau()	x 1	15 o sur place, dans main()	
textePanneau()	x 1	11 o sur place, dans main()	+ EcrireTexte (10 o)
defiler()	x 1	7 o sur place, dans main()	
image()	x 1	3 o sur place, dans main()	+ AttendreImage (28 o)

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
#include <Tuile>           // un dessin de 8 × 8 pixels
#include <poser>            // pose une tuile sur une case du fond
#include <textePanneau>     // écrit un texte sur le panneau
#include <panneau>         // montre le panneau, à une place choisie
#include <defiler>         // fait glisser tout le fond
#include <paletteFond>     // choisit les quatre nuances du fond
```

2. UN DESSIN — SOL

```
Tuile SOL = {
    "33333333",
    "32222223",
    "32122123",
    "32222223",
    "32221223",
    "32122223",
    "32222223",
    "33333333",
};
```



à l'écran

3. UNE VARIABLE — D

```
uint8_t d = 0;
```

4. UNE VARIABLE — ATTENTE

```
uint8_t attente = 0;
```

5. UNE VARIABLE — NOIRCEUR

```
uint8_t noirceur = 0;
```

6. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    for (uint8_t x = 0; x < 32; x++) {
        poser(x, 12, SOL);
        poser(x, 13, SOL);
    }

    textePanneau(1, 0, "SCORE 00");
    panneau(0, 128);

    while (true) {
        image();

        d++;
        defiler(d, 0);

        attente++;

        if (attente > 40) {
            attente = 0;
            noirceur++;
            if (noirceur > 3) noirceur = 0;

            // le fond s'assombrit, puis revient
            paletteFond(noirceur, noirceur + 1, 2, 3);
        }
    }

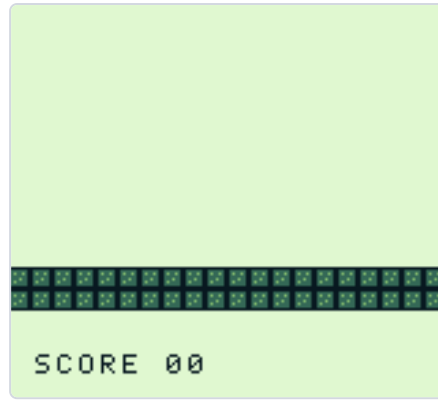
    return 0;
}
```

3. Ce qui se passe, étape par étape

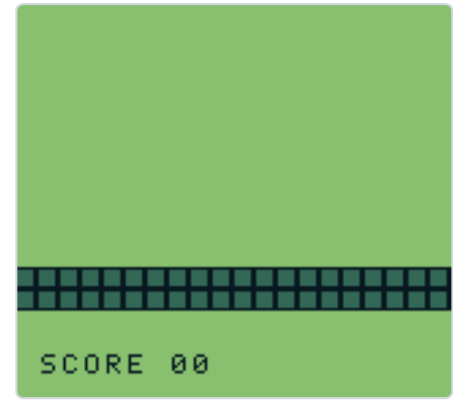
La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



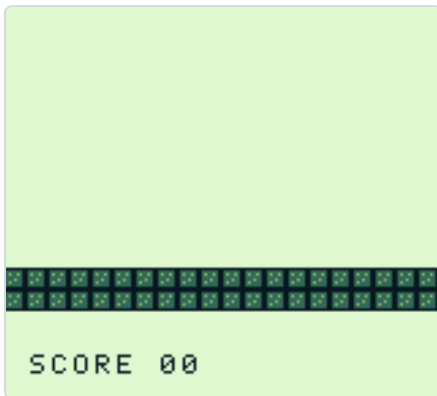
la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de trois secondes — image 180



au bout de sept secondes — image 420

4. Ce qu'on doit voir

Le sol glisse ; « SCORE 00 » reste collé en bas, et les nuances changent.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ le panneau est allumé
- ✓ il lit la seconde carte de fond
- ✓ il est posé à la ligne 128
- ✓ le décor défile
- ✓ mais le panneau ne bouge pas
- ✓ et il n'est pas vide
- ✓ les nuances ont changé (\$e4 vers \$ee)

Ensuite : **62. Remplir un tableau, puis le dessiner** — chapitre 9 : Ranger ce qu'on manipule. · gameboy3 — écrire une cartouche Game Boy en C++